

Margin Engine

Simplified Implementation Reference

`margin_engine_simplified.py`

Trade Republic Bank GmbH
Risk Modelling

April 2026

STRICTLY PRIVATE & CONFIDENTIAL

Contents

1 Overview	2
2 Return Decomposition and Calibration Rationale	2
3 GBM Monte Carlo Calibration	2
3.1 Single-Instrument Simulation (<code>_simulate_gbm</code>)	2
3.2 Batch Simulation (<code>simulate_gbm_batch</code>)	2
3.3 VaR Extraction and Rate Calibration (<code>_var_from_mc</code>)	3
4 SimplifiedMarginEngine Class	3
4.1 Constructor	3
4.2 Method 1: <code>calibrate(params)</code> — Calibrate all three rates	3
4.3 Method 2: <code>check_opening</code> — IM phase opening check	4
4.4 Method 3: <code>monitor</code> — Continuous LTV check	4
4.5 Method 4: <code>liquidation_price</code> — Auto-liquidation trigger price	5
4.6 Method 5: <code>position_summary</code> — Full one-row summary	5
5 Data Flow	5
6 Symbol Reference	6

1. Overview

`margin_engine_simplified.py` implements the simplified, rule-based margin framework described in `margin_methodology_simple.tex`. It exposes a single class `SimplifiedMarginEngine` and two standalone functions `simulate_gbm_batch` and `_var_from_mc`.

The central design principle is that **the margin rate is the LTV limit**:

$$\text{LTV}_t = \frac{L_t}{\text{MV}_t} \leq 1 - h_{\text{phase}}(t)$$

No separate Accepted Collateral Value (ACV) step is required. Three margin rates are maintained per instrument, each calibrated from the volatility of a distinct return window via GBM Monte Carlo:

Rate	Return window	Risk captured	Input vol
h_{im}	Close-to-close (C→C)	Overnight gap + full intraday move	<code>sigma_cc</code>
h_{mm}	Open-to-close (O→C)	Intraday move only (continuous monitoring)	<code>sigma_oc</code>
h_{on}	Close-to-open (C→O)	Overnight gap only (EOD add-on)	<code>sigma_co</code>

The LTV limits in each phase and check type are:

Phase	Check	LTV limit
IM ($t = 0$)	Intraday	$1 - h_{\text{im}}$
IM ($t = 0$)	Overnight	$1 - h_{\text{im}} - h_{\text{on}}$
MM ($t \geq 1$)	Intraday	$1 - h_{\text{mm}}$
MM ($t \geq 1$)	Overnight	$1 - h_{\text{mm}} - h_{\text{on}}$

2. Return Decomposition and Calibration Rationale

The close-to-close return decomposes additively into an overnight gap and an intraday component:

$$r_t^{C \rightarrow C} = r_t^{C \rightarrow O} + r_t^{O \rightarrow C}$$

At position open (*IM phase*), the customer is exposed to both components before the first monitoring check fires. The initial margin rate h_{im} must therefore cover the full C→C risk horizon.

Once in the *MM phase* ($t \geq 1$), monitoring is continuous from market open. The overnight gap has already materialised and is no longer a forward risk; only the intraday O→C component remains. The maintenance margin rate h_{mm} is therefore calibrated from O→C volatility and is structurally lower than h_{im} .

The overnight add-on h_{on} is applied at EOD to close the gap between intraday and overnight risk and is calibrated from C→O volatility.

In annualised terms the approximate variance identity is:

$$\sigma_{C \rightarrow C}^2 \approx \sigma_{C \rightarrow O}^2 + \sigma_{O \rightarrow C}^2 \implies h_{\text{im}} \gtrsim h_{\text{mm}} + h_{\text{on}}$$

3. GBM Monte Carlo Calibration

3.1 Single-Instrument Simulation (`_simulate_gbm`)

Each margin rate is derived from a GBM simulation of 1-day returns. For a given annualised drift μ and volatility σ , the 1-day log-return under Geometric Brownian Motion is:

$$\ln\left(\frac{S_1}{S_0}\right) = \left(\mu - \frac{1}{2}\sigma^2\right) dt + \sigma \sqrt{dt} Z, \quad Z \sim \mathcal{N}(0, 1), \quad dt = \frac{1}{252}$$

The simple return is recovered as $r = e^{\ln(S_1/S_0)} - 1$. The function draws N independent standard normal variates $Z_1, \dots, Z_N$, computes the corresponding returns, and returns them as an array.

3.2 Batch Simulation (`simulate_gbm_batch`)

`simulate_gbm_batch` accepts a `DataFrame` indexed by `instrument_id` with columns `mu`, `sigma`, `n_sim`, and optionally `seed`, and runs `_simulate_gbm` for each row.

The output is a DataFrame indexed by `instrument_id` with one column per simulation path ($0 \dots N_{\max} - 1$), where $N_{\max} = \max_i n_{\text{sim}_i}$. Instruments with fewer paths receive NaN padding so all rows share the same width.

```
params = pd.DataFrame({
    "mu":      [0.05, 0.08],
    "sigma":   [0.20, 0.35],
    "n_sim":   [10_000, 10_000],
    "seed":    [42, 42],
}, index=["AAPL", "TSLA"])

sim = simulate_gbm_batch(params)
# sim.loc["TSLA"] -> array of 10,000 simulated 1-day simple returns
```

3.3 VaR Extraction and Rate Calibration (`_var_from_mc`)

The margin rate for a given return window is the worst-case 1-day loss at confidence level α , floored at a minimum rate h_{floor} :

$$h = \max(h_{\text{floor}}, -\text{quantile}(r_{1:N}, 1 - \alpha))$$

The negation of the $(1 - \alpha)$ -quantile gives a positive loss magnitude. Used directly as the margin rate, it ensures that a position opened at the maximum permissible LTV will — in the worst simulated scenario at confidence α — still leave the customer's equity non-negative.

The `floor_binding` flag in the output is `True` when the floor raised the rate above the raw MC quantile, indicating a floor-constrained (rather than vol-driven) result.

Output field	Description
<code>h_{label}</code>	Final margin rate after flooring
<code>var_id_{label}</code>	Raw MC quantile before flooring
<code>sigma_id_{label}</code>	Annualised σ scaled to 1-day: $\sigma/\sqrt{252}$
<code>floor_binding_{label}</code>	True if floor exceeded the raw VaR

4. SimplifiedMarginEngine Class

4.1 Constructor

```
engine = SimplifiedMarginEngine(
    confidence = 0.99,      # VaR confidence level for all three calibrations
    n_sim      = 10_000,    # GBM paths per instrument
    seed       = 42,        # RNG seed; None for non-deterministic
)
```

4.2 Method 1: `calibrate(params)` — Calibrate all three rates

Runs three independent GBM calibrations per instrument ($C \rightarrow C$, $O \rightarrow C$, $C \rightarrow O$) and returns a summary DataFrame.

Required input columns:

Column	Type	Description
<code>sigma_cc</code>	float	Annualised $C \rightarrow C$ volatility
<code>sigma_oc</code>	float	Annualised $O \rightarrow C$ volatility
<code>sigma_co</code>	float	Annualised $C \rightarrow O$ volatility

Optional input columns (defaults in parentheses):

Column	Default	Description
<code>mu_cc</code>	0.0	Annualised drift for $C \rightarrow C$ simulation
<code>mu_oc</code>	0.0	Annualised drift for $O \rightarrow C$ simulation
<code>mu_co</code>	0.0	Annualised drift for $C \rightarrow O$ simulation
<code>h_floor_im</code>	0.10	Minimum IM rate
<code>h_floor_mm</code>	0.05	Minimum MM rate
<code>h_floor_on</code>	0.02	Minimum overnight add-on

Output columns (one row per instrument):

Column	Description
h_im, h_mm, h_on	Calibrated margin rates
var_1d_im, var_1d_mm, var_1d_on	Raw MC VaR before flooring
floor_binding_im/mm/on	Floor constraint flags
ltv_limit_im, ltv_limit_mm	$1 - h_{im}, 1 - h_{mm}$
ltv_limit_im_overnight	$1 - h_{im} - h_{on}$
ltv_limit_mm_overnight	$1 - h_{mm} - h_{on}$
max_leverage_intraday	$1/h_{im}$
max_leverage_overnight	$1/(h_{im} + h_{on})$

```

params = pd.DataFrame({
    "sigma_cc": [0.20, 0.35],
    "sigma_oc": [0.15, 0.27],
    "sigma_co": [0.13, 0.22],
    "h_floor_im": [0.10, 0.20],
}, index=["AAPL", "TSLA"])

rates = engine.calibrate(params)
# rates.loc["TSLA", "h_im"] -> e.g. 0.2234

```

Deriving sigma inputs from a price DataFrame:

```

# prices: DataFrame with columns "open" and "close", one row per date
cc = prices["close"].pct_change()
oc = prices["close"] / prices["open"] - 1
co = prices["open"] / prices["close"].shift(1) - 1

params["sigma_cc"] = cc.std() * np.sqrt(252)
params["sigma_oc"] = oc.std() * np.sqrt(252)
params["sigma_co"] = co.std() * np.sqrt(252)

```

4.3 Method 2: check_opening — IM phase opening check

Checks whether a new position satisfies the IM condition and, optionally, the overnight condition at the time of opening.

$$LTV_0 = \frac{L_0}{MV_0} \leq 1 - h_{im} \quad (\text{intraday})$$

$$LTV_0 \leq 1 - h_{im} - h_{on} \quad (\text{overnight})$$

Key output fields:

Field	Description
ltv	Current LTV = L/MV
passes_im	True if intraday IM condition satisfied
min_equity	$h_{im} \times MV$
max_loan_intraday	$(1 - h_{im}) \times MV$
max_leverage_intraday	$1/h_{im}$
passes_overnight	True if overnight condition satisfied
max_leverage_overnight	$1/(h_{im} + h_{on})$
topup_required	Cash to post to clear the overnight condition

```

check = engine.check_opening(
    market_value = 15_000,
    loan         = 10_000,
    h_im         = rates.loc["TSLA", "h_im"],
    h_on         = rates.loc["TSLA", "h_on"],
)
# check["passes_im"] -> True / False
# check["topup_required"] -> cash needed to satisfy overnight condition

```

4.4 Method 3: monitor — Continuous LTV check

Called at every price tick (MTM is continuous). Returns a PositionStatus dataclass with breach and shortfall.

$$h_{\text{eff}} = h_{\text{phase}} + h_{\text{on}} \cdot \mathbf{1}_{\text{overnight}}$$

$$\text{shortfall} = \max(0, L - (1 - h_{\text{eff}}) \times \text{MV})$$

The shortfall is the cash the customer must post to bring the LTV exactly to the applicable limit at the current market price. When `breach = True` and no cash is posted, auto-liquidation fires immediately.

```
# Intraday MM check
status = engine.monitor(
    market_value = 13_500,
    loan         = 10_000,
    h_phase      = rates.loc["TSLA", "h_mm"],
    h_on         = rates.loc["TSLA", "h_on"],
    phase        = "mm",
    overnight    = False,
)
print(status)
# [OK] LTV=74.1% limit=75.0% phase=mm shortfall=0.00

# EOD overnight check (same call, overnight=True)
status_on = engine.monitor(
    market_value = 13_500, loan = 10_000,
    h_phase      = rates.loc["TSLA", "h_mm"],
    h_on         = rates.loc["TSLA", "h_on"],
    phase        = "mm", overnight = True,
)
```

4.5 Method 4: `liquidation_price` — Auto-liquidation trigger price

Returns the price P^* at which LTV equals the applicable limit:

$$P^* = \frac{L}{N \times (1 - h_{\text{eff}})}$$

```
p_star = engine.liquidation_price(
    loan      = 10_000,
    n_units   = 150,
    h_phase    = rates.loc["TSLA", "h_mm"],
    h_on       = rates.loc["TSLA", "h_on"],
    overnight  = False,
)
# e.g. 88.89 -- price below which auto-liquidation triggers in MM phase
```

4.6 Method 5: `position_summary` — Full one-row summary

Produces a single-row DataFrame with all LTV checks and liquidation prices for all four phase/overnight combinations.

```
summary = engine.position_summary(
    market_value = 15_000,
    loan         = 10_000,
    n_units      = 150,
    rates        = rates.loc["TSLA"],
)
# Columns: h_im, ltv_limit_im, passes_im, liq_price_im,
#          ltv_limit_im_overnight, passes_im_overnight, liq_price_im_overnight,
#          h_mm, ltv_limit_mm, passes_mm, liq_price_mm,
#          ltv_limit_mm_overnight, passes_mm_overnight, liq_price_mm_overnight,
#          h_on, max_leverage_intraday, max_leverage_overnight
```

5. Data Flow

1. **Compute volatilities** from a price DataFrame with `open` and `close` columns using $C \rightarrow C$, $O \rightarrow C$ and $C \rightarrow O$ return series.
2. **Calibrate** via `engine.calibrate(params)` to obtain `h_im`, `h_mm`, `h_on` per instrument.
3. **At position open**: call `check_opening` to verify the IM and overnight conditions. Reject if `passes_im = False`.
4. **Intraday (day 0)**: call `monitor` with `phase="im"` and `overnight=False` at each price update.
5. **EOD (day 0)**: call `monitor` with `overnight=True`. If `breach=True`: require customer top-up of `shortfall`.

6. **From day 1 onwards:** repeat steps 4–5 with `phase="mm"`. The MM LTV limit is wider; the overnight check uses the same add-on.
7. **Auto-liquidation:** triggered immediately when `breach=True` and no cash is posted. No margin call is issued.

6. Symbol Reference

Symbol	Code field	Description
L_t	<code>loan</code>	Outstanding loan at time t
MV_t	<code>market_value</code>	Market value = $N \times P_t$
LTV_t	<code>ltv</code>	L_t/MV_t
h_{im}	<code>h_im</code>	Initial margin rate (C→C calibrated)
h_{mm}	<code>h_mm</code>	Maintenance margin rate (O→C calibrated)
h_{on}	<code>h_on</code>	Overnight add-on (C→O calibrated)
h_{eff}	—	$h_{phase} + h_{on} \cdot \mathbf{1}_{overnight}$
σ_{CC}	<code>sigma_cc</code>	Annualised C→C volatility
σ_{OC}	<code>sigma_oc</code>	Annualised O→C volatility
σ_{CO}	<code>sigma_co</code>	Annualised C→O volatility
α	<code>confidence</code>	VaR confidence level (default 0.99)
N	<code>n_sim</code>	Number of GBM simulation paths
P^*	<code>liq_price_*</code>	Auto-liquidation trigger price
dt	—	1/252 (one trading day)